

LAPORAN PRAKTIKUM

TEKNOLOGI IOT 2

Fondasi Arsitektur Internet of Things (IoT) dan Mikrokontroler



Disusun Oleh :

Muhamad Ramadhani (25782045)

Dosen Pembimbing :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T..

Agus Ambarwari, S.Pd., M.Kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

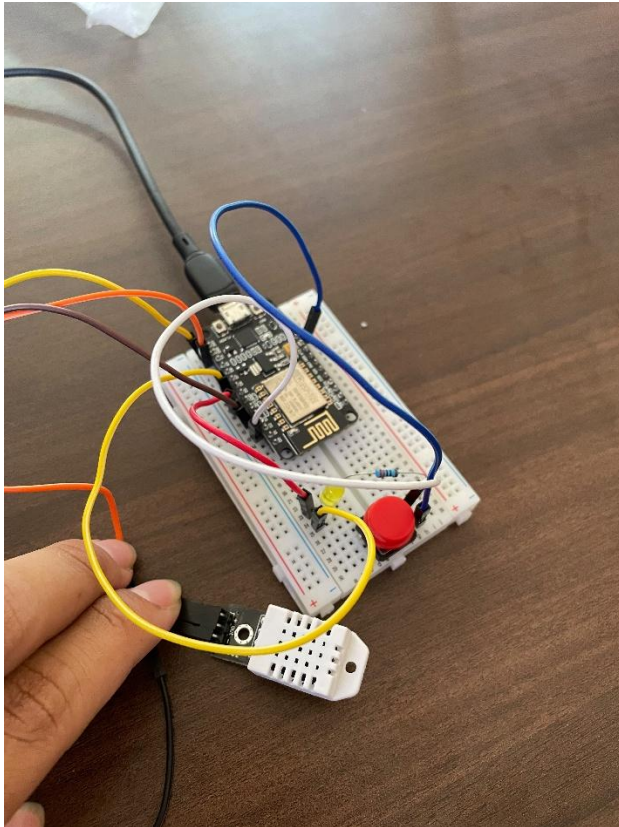
4.5 Praktikum

4.5.1 Praktikum 1: *Real-Time Dashboard & Sinkronisasi Tombol Fisik*

Skenario: Anda memiliki lampu LED (simulasi aktuator/lampu cerdas) yang bisa dihidupkan melalui sakelar fisik (*push button*) di *breadboard*, ATAU melalui tombol di antarmuka web *smartphone* Anda. Keduanya harus tersinkronisasi dua arah secara *real-time*. Jika Anda menekan tombol fisik di meja praktikum, status visual indikator di antarmuka *smartphone* harus otomatis berubah seketika, begitu pula sebaliknya jika dikontrol dari web.

Langkah Kerja Rangkaian:

1. Hubungkan kaki **Anoda** (kaki panjang) LED ke pin **D6 (GPIO 12)**. Hubungkan kaki **Katoda** (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke pin **GND** NodeMCU (*sama seperti rangkaian LED pada Modul 3*).
2. Hubungkan pin **DATA / OUT** DHT ke pin **D4 (GPIO 2)**. (VCC DHT ke 3V3, GND ke GND).
3. Pasang **Push Button** pada *breadboard*, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur **3V3 (3.3V)**. Hubungkan kaki seberang tombol ke pin **D2 (GPIO 4)** NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki **Resistor 10k Ohm**, lalu hubungkan ujung kaki resistor lainnya ke jalur **GND** (*konfigurasi resistor pull-down eksternal seperti pada Modul 1*).



Langkah Kerja Pemrograman: Kode program ini cukup kompleks karena mengombinasikan *Front-End* (HTML, CSS, JavaScript) dengan *Back-End* (C++). Agar *real-time response* tombol tidak terganggu oleh sensor DHT yang butuh jeda baca lama, kita menerapkan teknik pemrograman *non-blocking* menggunakan fungsi `millis()`.

1. Buat berkas baru (*New Sketch*). Tuliskan kode program berikut.

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>

const char* ssid = "NAMA_WIFI_ANDA";
const char* password = "PASSWORD_WIFI_ANDA";

// Konfigurasi Pin
const byte dhtPin = 2;          // D4 (GPIO 2)
const byte buttonPin = 4;       // D2 (GPIO 4)
const byte ledPin = 12;         // D6 (GPIO 12)

DHT dht(dhtPin, DHT11);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;
```

```
// Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// ----- HTML & JAVASCRIPT (FRONT-END) -----
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Real-Time IoT Web</title>
  <style>
    body { font-family: Arial; text-align: center; }
    .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-width: 300px; border-radius: 10px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 5px; cursor: pointer; color: white; }
    .btn-on { background-color: #4CAF50; }
    .btn-off { background-color: #f44336; }
  </style>
</head>
```

```

<div class="card">
  <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
</div>
<div class="card">
  <h2>LED: <span id="ledStatus">OFF</span></h2>
  <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
</div>

<script>
  // Membuka terowongan koneksi WebSocket ke alamat IP ESP8266
  var gateway = `ws://${window.location.hostname}/ws`;
  var websocket;

  // Menjalankan inisiasi saat halaman pertama kali dimuat
  window.addEventListener('load', onLoad);
  function onLoad(event) { initWebSocket(); }

  function initWebSocket() {
    websocket = new WebSocket(gateway);
    websocket.onopen = onOpen;
    websocket.onclose = onClose;
    websocket.onmessage = onMessage;
  }

  function onOpen(event) { console.log('WebSocket Terkoneksi'); }
  function onClose(event) { setTimeout(initWebSocket, 2000); }

  // Dipanggil saat tombol di layar web ditekan
  function toggleLed(){
    websocket.send('toggle');
  }

  // Menangkap paket data JSON yang "didorong" (PUSH) oleh C++ NodeMCU
  function onMessage(event) {
    var dataObj = JSON.parse(event.data);

    // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
    if(dataObj.suhu !== undefined) {
      document.getElementById('tempValue').innerHTML = dataObj.suhu;
    }

    // Merombak UI Tombol & Teks sesuai status hardware
    if(dataObj.led !== undefined) {
      var btn = document.getElementById('toggleBtn');
      var status = document.getElementById('ledStatus');
      if(dataObj.led == "1"){
        status.innerHTML = "ON";
        btn.innerHTML = "Turn OFF";
        btn.className = "btn-on";
      }
    }
  }

```

```
// ----- BACK-END & WEBSOCKET LOGIC -----

// Fungsi menyebarkan paket data JSON ke SELURUH browser pengunjung
void notifyClients() {
    String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
    jsonString += "\"suhu\":\"" + currentTemp + "\"}";
    ws.textAll(jsonString);
}

// Handler pesan masuk (dari klik browser)
void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
    AwsFrameInfo *info = (AwsFrameInfo*)arg;
    if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT) {
        data[len] = 0;
        if (strcmp((char*)data, "toggle") == 0) {
            ledState = !ledState;
            notifyClients();
        }
    }
}

// Event handler WebSocket bawaan AsyncWebServer
void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client, AwsEventType type,
             void *arg, uint8_t *data, size_t len) {
    switch (type) {
```

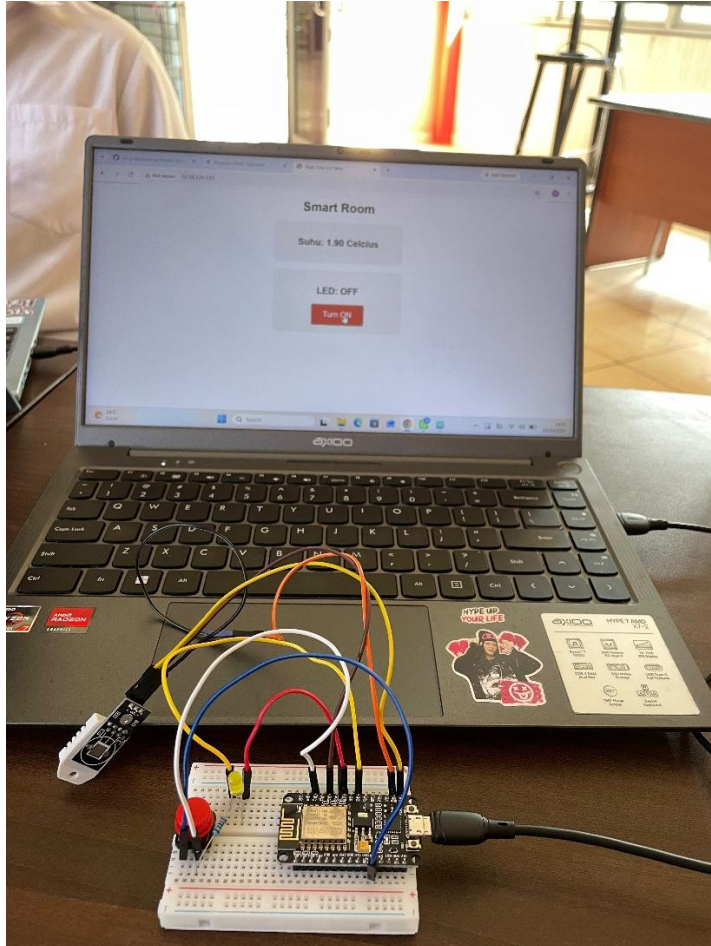
2. **Upload** program. Pastikan IP Address didapatkan dari Serial Monitor.
3. Buka IP Address tersebut di *browser* laptop Anda, dan **sekalius** buka IP yang sama di *browser* ponsel (*smartphone*) Anda secara berdampingan.
4. **Uji 1 (Kendali Web Tersinkronisasi):** Klik tombol pada *browser* laptop Anda. Perhatikan bagaimana *browser* di ponsel Anda ikut berubah (*Turn ON*) secara presisi, dan lampu LED fisik ikut menyala!
5. **Uji 2 (Kendali Perangkat Keras):** Sentuh *push button* mekanik di meja Anda. Amati bahwa antarmuka *browser* di laptop maupun ponsel Anda secara serentak berganti status (*Turn OFF* menjadi *Turn ON*) **tanpa** terjadi proses *loading* atau *refresh* HTML halaman sama sekali! Kecepatan respon seketika ini adalah manfaat dari pemrograman *Asynchronous* dan pemisahan (*decoupling*) pembacaan memori sensor DHT.

Penjelasan Singkat Kode Khusus:

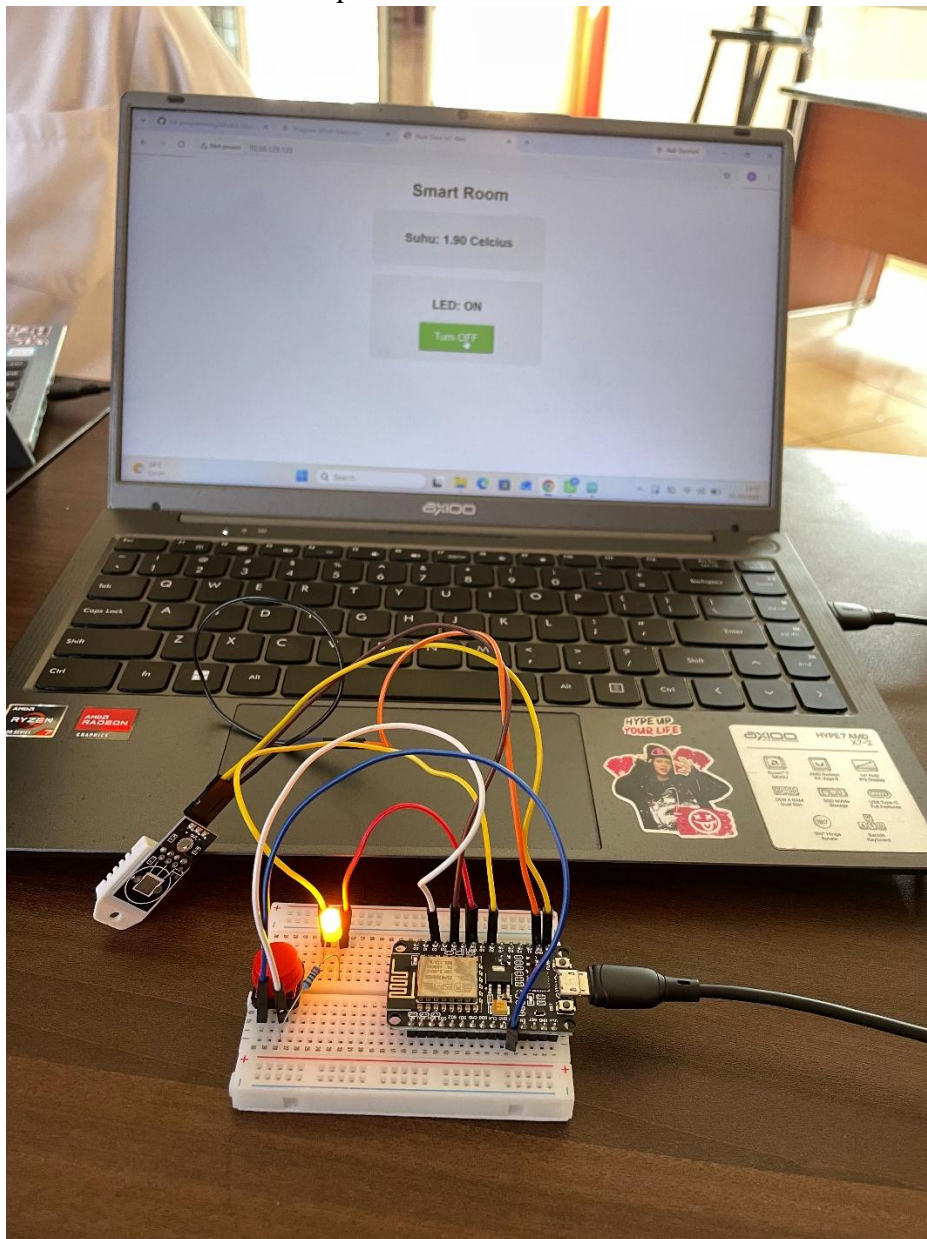
- `ws.cleanupClients()`: (Berada di siklus `loop()`). Fungsi penting dari pustaka *Asynchronous* ini bertugas membersihkan dan menutup sesi (*resource memori*) dari klien/*browser* yang sudah terputus atau menutup tab layarnya. Tanpa baris ini, memori RAM ESP8266 akan penuh (*memory leak*) dan berujung pada *Crash/Restart* jika terlalu banyak perangkat keluar-masuk membuka halaman *Dashboard*.
- `pinMode(buttonPin, INPUT)` dan `if (buttonState == HIGH)`: Mempertahankan konfigurasi resistor *Pull-Down* eksternal 10k Ohm ke jalur GND (seperti pada Modul 1). Saat tombol dilepas, pin D2 membaca logika **LOW** (0V). Saat tombol ditekan, arus 3.3V mengalir sehingga terbaca logika **HIGH** yang memicu pembalikan status (*toggle*) LED secara instan.
- `if ((millis() - lastTime) > 3000)`: Pemrograman *non-blocking*. Alih-alih membuat CPU tidur selama 3 detik menggunakan `delay(3000)` (yang akan membekukan respons WebSocket), kita menggunakan selisih penghitung jam internal CPU (`millis()`). Dengan demikian, CPU dapat terus siaga mengawasi sentuhan tombol fisik dan interaksi jaringan internet dengan kecepatan tinggi.

- `var dataObj = JSON.parse(event.data)`: Sintaks JavaScript yang membedah teks kaku kiriman C++ berformat `{"suhu": "30", "led": "1"}` menjadi struktur data objek dinamis (*Dictionary*) untuk memperbarui elemen desain HTML dengan instan (*DOM Manipulation*).
- `ws.textAll()`: Perintah penyiaran (*broadcasting*) ampuh yang menembakkan/menyebarkan data secara seketika ke *seluruh* perangkat *browser* pengunjung (laptop, HP) secara bersamaan.

Kondisi LED jika kondisi lagi mati



Kondisi Ketika LED hidup



4.6 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda terhadap arsitektur web modern!

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawab: WebSockets lebih hemat bandwidth dan tidak membuat layar berkedip karena koneksi antara browser dan server tetap terbuka (persistent connection). Setelah koneksi dibuat, server dapat langsung mengirimkan data terbaru ke browser hanya ketika ada perubahan atau data baru, tanpa harus membuat permintaan HTTP berulang-ulang.

Sedangkan pada metode Meta Refresh HTTP, browser akan melakukan request HTTP baru secara berkala untuk mengambil seluruh halaman HTML kembali. Hal ini menyebabkan lebih banyak data dikirim melalui jaringan dan browser harus memuat ulang halaman, sehingga layar dapat terlihat berkedip.

Jadi, perbedaannya adalah:

- Meta Refresh HTTP: halaman dimuat ulang berkali-kali → lebih banyak request dan bandwidth → layar berkedip.
 - WebSockets: satu koneksi tetap terbuka → hanya data yang berubah yang dikirim → lebih hemat bandwidth dan tampilan lebih lancar.
- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Jawab: `delay(3000)` membuat NodeMCU berhenti selama 3 detik sehingga WebSocket tidak dapat memproses komunikasi secara terus-menerus. Akibatnya, respon browser menjadi lambat, data sensor terlambat dikirim, dan koneksi WebSocket dapat terganggu atau terputus. Karena itu, sebaiknya gunakan timer seperti `millis()` agar komunikasi tetap responsif.

Latihan 3 (Modifikasi Front-End): Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

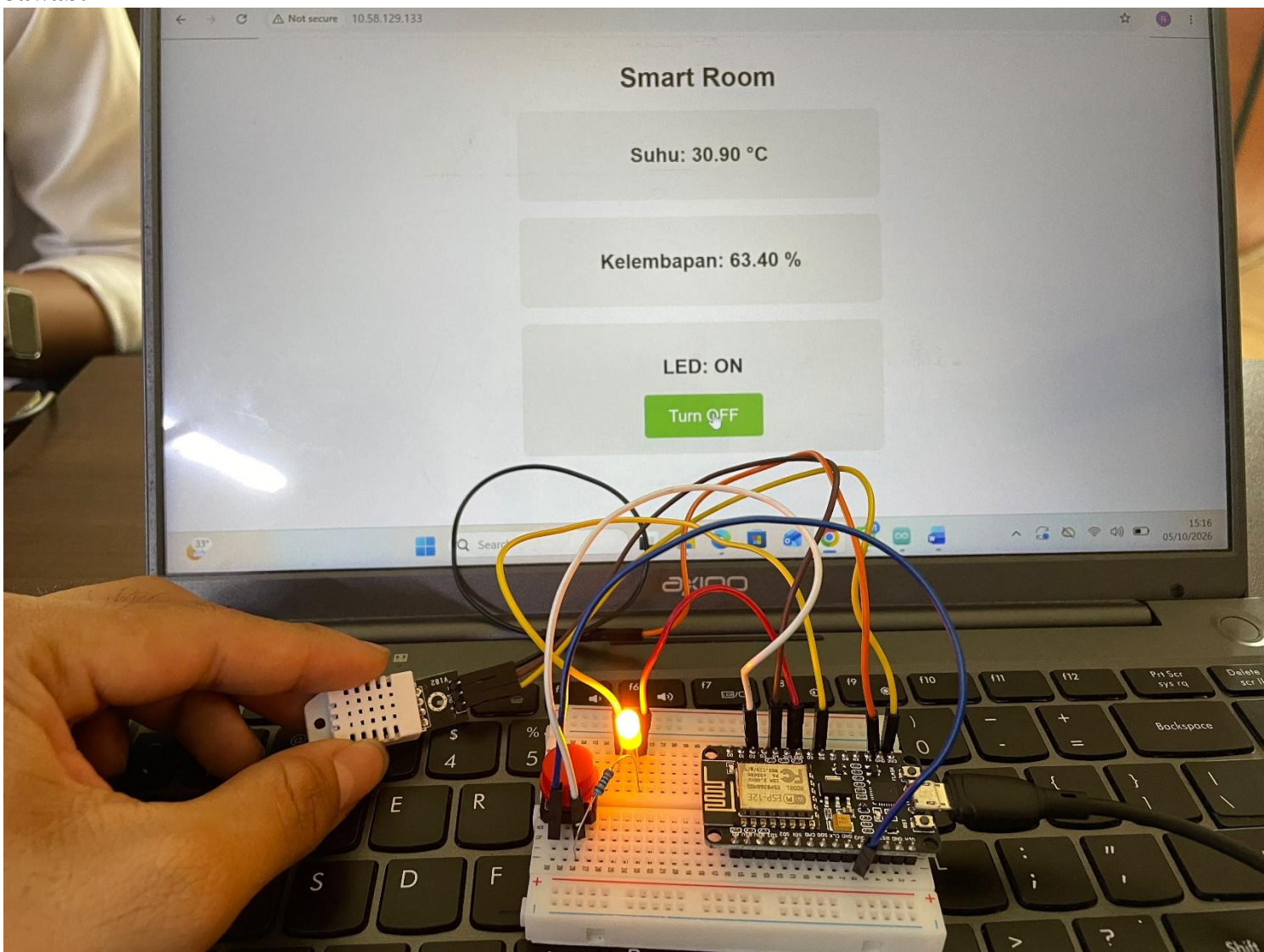
Jawab:

ID elemen HTML yang menjadi target perubahan oleh JavaScript adalah:

- `tempValue` → menampilkan nilai suhu.
- `ledStatus` → menampilkan status LED.
- `toggleBtn` → mengubah teks/status to

- **Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:
 - Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--");. Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led":"...", "suhu":"...", "hum":"..."}`).
 - Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen `humValue`. Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawab:



4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*

Sinyal Digital (*ON/OFF*) sangat berguna, namun bagaimana jika Anda ditugaskan merancang antarmuka penyesuaian intensitas redup-terang lampu (*Dimmer*) dari *smartphone*?

Skenario Proyek (*Smart Lighting*): Anda akan mengeksplorasi penggunaan modulasi sinyal listrik analog `Write()` (atau dikenal dengan fitur PWM) dan memadukannya dengan teknologi masukan elemen *Slider / Range* dari HTML5.

- **Tujuan:** Menguasai transmisi paket *String* WebSocket berkelanjutan (*continuous packet*) dari manipulasi antarmuka layar *slider* untuk merombak parameter persentase daya komponen aktuator.
- **Tingkat Kesulitan:** Lanjut
- **Instruksi Perangkaian:**
 - i. Tambahkan sebuah komponen **LED** yang dilengkapi **Resistor 220 Ohm** (sebagai pengaman arus).
 - ii. Hubungkan kaki Anoda (positif) sistem LED tersebut ke pin **D1 (GPIO 5)**.
- **Instruksi Pemrograman (Front-End & Back-End):**
 - i. Modifikasi desain kerangka web HTML Anda. Sisipkan elemen input *Slider* HTML. (*Hint sintaksis: `<input type="range" min="0" max="1023" id="pwmSlider" onchange="sendPWM(this.value)">`*)
 - ii. Buat sebuah fungsi JavaScript bernama `sendPWM(value)` yang akan mengirimkan paket *string* teks berformat khusus (misalnya "pwm,512") ke NodeMCU melalui *websocket*.
 - iii. Pada ruang fungsi C++ `handleWebSocketMessage()` di sisi server (NodeMCU), pecah (*parsing*) atau periksa string tersebut menggunakan fitur deteksi *substring* atau bahasa pemrosesan C++ lainnya.
 - iv. Jika Server NodeMCU sukses mendeteksi dan mengambil angka murninya, panggil fungsi aktuator `analogWrite(pinLED, nilaiPWM)` untuk secara elegan meredupkan atau menerangkan LED LED fisik secara seketika sesuai gesekan layar Anda!
- **Kriteria Keberhasilan:**
 - Di layar gawai Anda, hadir tuas gulir horizontal (*Slider*).
 - LED di dunia nyata dapat diredupkan dan diterangkan secara bertahap saat Anda menggeser tuas *slider*, bebas dari lag (*real-time response*).
 - Kendali LED dan pembacaan Sensor *DHT* bawaan Praktikum 1 dipastikan masih bekerja berdampingan dengan sempurna.

[PENTING!]

Instruksi Pengumpulan Tugas Praktikum Seluruh pengerjaan praktikum dan tugas (meliputi *source code* berformat .ino, dokumentasi foto/video sirkuit yang berhasil, tangkapan layar *Serial Monitor*, serta laporan praktikum tertulis .PDF) harus dikumpulkan dengan cara melakukan **commit dan push ke repositori GitHub pribadi Anda masing-masing**.

Tautkan/kumpulkan *URL repositori GitHub* Anda pada sistem manajemen pembelajaran (LMS) kampus sebagai bukti penyelesaian praktikum ini.

Hasil dari tugas



Disini menunjukan hasil jika intensitas semakin tinggi maka lampu led akan semakin terang menyala dan begitu juga sebaliknya jika intensitas rendah maka lampunya juga akan semakin redup

